

1. Tech

Framework

Laravel 10

Pourquoi choisir Laravel ?

1. Syntaxe et structure élégantes :
Laravel offre un code clair, lisible et expressif qui rend le développement plus rapide et agréable.
2. Développement rapide :
Les fonctionnalités intégrées comme l'authentification, le routage, le cache, les files d'attente et la gestion des emails réduisent le travail répétitif.
3. Écosystème solide :
Laravel est livré avec Forge, Vapor, Nova, Sail, Horizon, etc., qui étendent les possibilités de déploiement, d'applications serverless, de tableaux de bord administratifs et de gestion des files.
4. Communauté et support :
L'un des plus grands frameworks PHP, avec une vaste documentation, des tutoriels et des packages. Si vous rencontrez un problème, il y a de fortes chances que quelqu'un l'ait déjà résolu.
5. Architecture MVC :
Encourage la séparation des responsabilités, ce qui rend les applications plus faciles à maintenir et à faire évoluer.
6. Moteur de templates Blade :
Simple mais puissant pour créer des composants UI dynamiques et réutilisables.
7. Fonctionnalités de sécurité :
Protection CSRF intégrée, mots de passe hachés, chiffrement et flux d'authentification sécurisés.

(Serveur) Prérequis

- PHP ^8.1
 - Composer
 - Node.js & NPM
 - MySQL/Postgres (ou votre base de données préférée)
 - Laravel 10
 - Laravel Livewire ^3.x
 - MySQL v5.7+
- Plus d'infos : <https://laravel.com/docs/10.x/deployment#server-requirements>

Installation

1. Cloner le dépôt :

```
git clone https://github.com/Telraam-Rear-Window-BV/ecomobilis.git
cd your-project
```

2. Installer les dépendances PHP : `composer install`
3. Installer les dépendances front-end : voir ci-dessous
4. Copier le fichier d'environnement et configurer :

```
Copy code
cp .env.example .env
php artisan key:generate
```

5. Importer la base de données (resources/setup-files-db-etc/full-db-structure-2025-09-25.sql)

Frontend (Laravel mix)

- Où éditer : utiliser les fichiers `resources/js/main.js` et `resources/sass/...`
- Les chemins et fichiers CSS/JS sont définis dans `webpack.mix.js` + config avec `package.json`
- Compiler avec : `npm run watch` / `npm run production`

Jobs

Pour le démarrer en local :

- `php artisan queue:listen database --tries=1`

Commandes

- `php artisan CarpoolSendUserUpcomingRides:daily` Ceci s'exécute via une tâche cron sur le serveur (* * * * * `php /data/sites/web/ecomobilisbe/laravelproject/artisan schedule:run`)

Comment obtenir les rues ?

<https://locationiq.com/demo#autocomplete> -> utilise OpenStreetMap

Paquets JS

- Package JS Autocomplete utilisé pour la recherche de rues : <https://github.com/TarekRaafat/autoComplete.js>
- Leaflet pour les cartes, package open source basé sur OpenStreetMap

Paquets Laravel

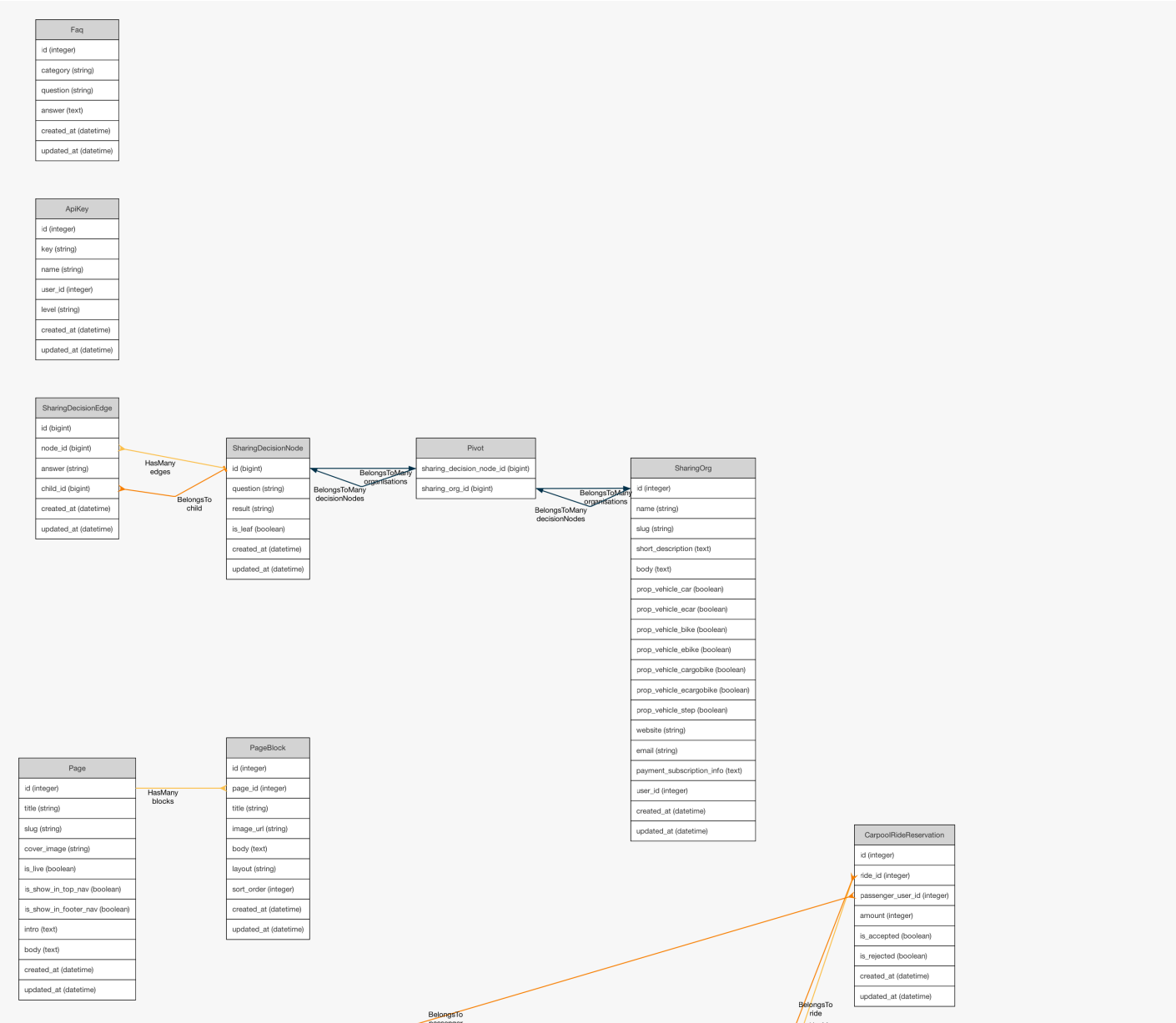
<https://github.com/msurguy/Honeypot/tree/master> Laravel ER Diagram Generator

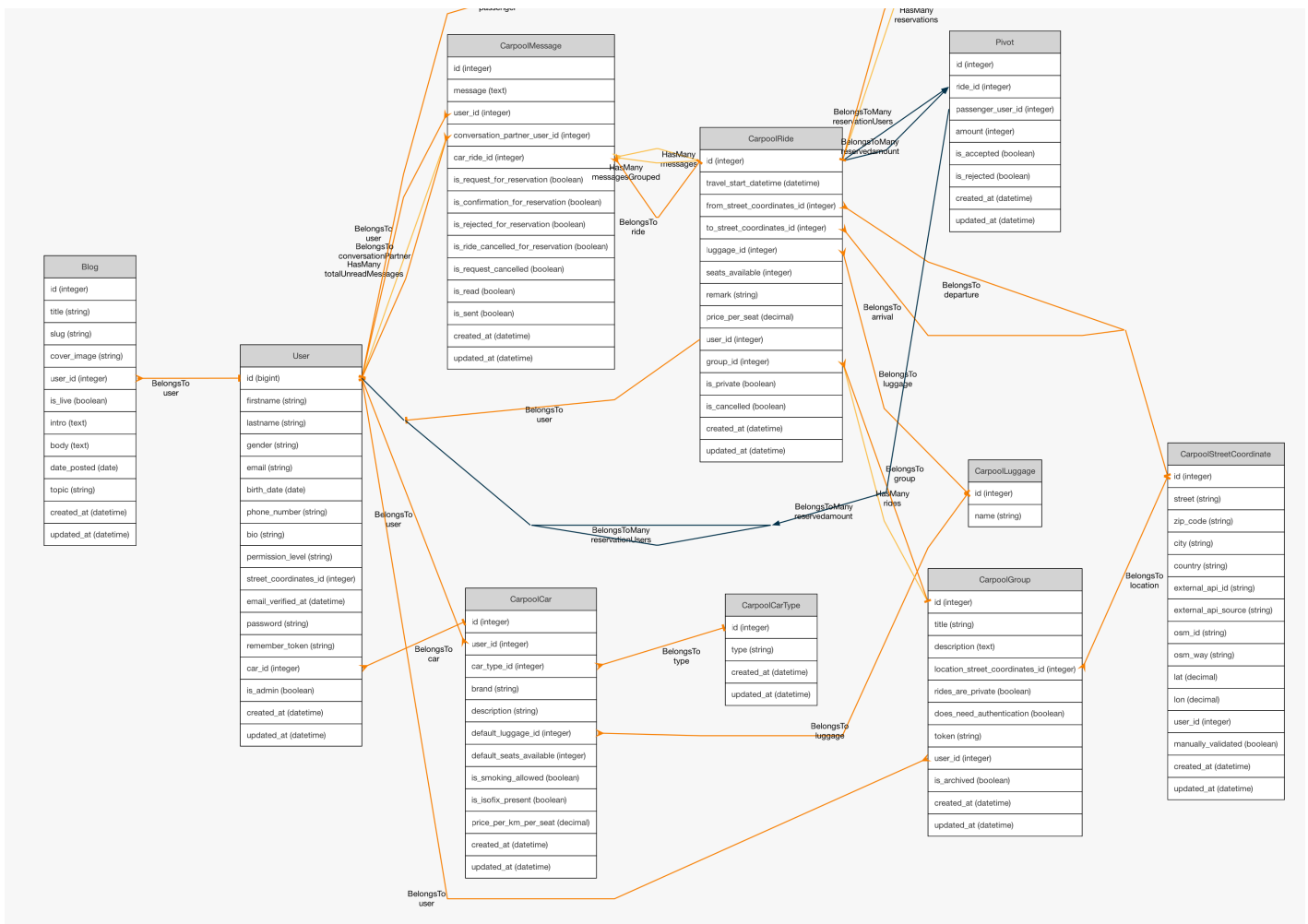
Laravel Livewire

Ce projet utilise Laravel Livewire pour les composants réactifs. Flux de travail typique : Créer des composants avec `php artisan make:livewire ExampleComponent` Les composants se trouvent dans `app/Http/Livewire/` avec les vues Blade correspondantes dans `resources/views/livewire/`. Utiliser les composants dans les templates Blade avec : `<livewire:example-component />`

2. Base de données

Au départ, nous envisagions une base de données NoSQL — car la structure semblait devoir être très flexible — mais nous avons choisi MySQL. Les modules de covoiturage développés bénéficient des relations entre tables. Les fonctionnalités relationnelles (clés étrangères, jointures, ...) garantissent l'intégrité des données, ce qui manque souvent dans le NoSQL. Presque tous les hébergeurs supportent MySQL par défaut, qui est gratuit, open source et largement maîtrisé.





(Generated with <https://github.com/beyondcode/laravel-er-diagram-generator>)

Les mises à jour ne se font pas actuellement via des migrations mais via des commandes SQL documentées. Elles se trouvent dans `resources/setup-files-db-etc/queries.sql`. La structure de base actuelle de la base est aussi documentée dans `resources/setup-files-db-etc/`.

3. Covoiturage

Localisations

Pour obtenir des localisations, nous utilisons l'API externe LocationIQ. Ce service utilise OpenStreetMap et permet de rechercher des rues, villes, etc. Il convertit une adresse structurée ou libre en coordonnées géographiques. Nous avons cherché des services totalement gratuits et ouverts, mais il n'y en a pas. LocationIQ semblait adapté et propose une tarification claire. Il est gratuit pour 5000 requêtes/jour et 2 requêtes/sec. Lorsqu'une localisation est soumise, nous stockons les coordonnées dans la table `carpool_street_coordinates` pour pouvoir les référencer ultérieurement. Après comparaison, LocationIQ était l'option la plus adaptée.

Autocomplétion des villes

L'interface pour la sélection des localisations (le menu déroulant) utilise la librairie open source vanilla JavaScript : <https://github.com/TarekRaafat/autoComplete.js>

Nous stockons le JSON dans un champ caché sous le champ d'entrée. C'est ce champ caché qui est utilisé en base de données.

Calcul du prix

Lorsqu'un prix est calculé, nous utilisons la "distance en bref" + 20 %. Nous le multiplions par le prix au km fourni par l'utilisateur.

Messages

Un fil de messages est toujours lié à un trajet spécifique. Le fil se forme en combinant l'expéditeur (`user_id`), le destinataire (`conversation_partner_user_id`) et le trajet (`car_ride_id`).

Il existe un type particulier de message, le message automatique, déclenché par une requête externe (ex. demande de covoiturage). Ce message automatique apparaît dans le fil avec un texte fixe et une mise en page légèrement différente.

Emails

- Lorsqu'une réservation est demandée (événement `CarpoolRequested`)
- Lorsqu'une réservation est confirmée (`CarpoolReservationAccepted`)

- Lorsqu'une réservation est rejetée (CarpoolReservationRejected)
- Lorsqu'une conversation a lieu (CarpoolMessageAdded)
- Chaque matin, un aperçu des trajets à venir (Console/Kernel - carpoolSendUserUpcomingRides:daily 06:00)

Correspondance des covoiturages

Le matching est basé sur les coordonnées GPS enregistrées et recherche dans un rayon de 20 km autour du départ et de l'arrivée, en tenant compte de la date/heure donnée ou ultérieure.

4. API

L'API est ouverte et documentée : <https://documenter.getpostman.com/view/12029054/2sB3HrnHiF#6039ffb3-d303-4833-b58c-43d088177f13>

Les données Telraam (données relatives au trafic) sont également ouvertes et documentées: <https://telraam-api.net/>

5. Structure du projet

Laravel est un framework MVC avancé. La structure par défaut a été suivie.

6. Hébergement

L'hébergement de <http://ecomobilis.be/> est assuré chez Combell (combell.com). Un accès (S)FTP peut être fourni, il suffit d'envoyer un email à dave@telraam.net

7. Déploiement

Vérifiez que .env est configuré pour la production.

```
composer install --optimize-autoloader --no-dev
php artisan config:cache
php artisan route:cache
php artisan view:cache
npm run build
```

8. Design

Fonts

- DM Sans comme font principale (<https://fonts.google.com/specimen/DM+Sans>)
- Barlow Condensed pour les en-têtes étroits (<https://fonts.google.com/specimen/Barlow+Condensed>)

Palette de couleurs

Bleu foncé principal : #263d5e Orange accent : #F2845C Beige de fond : #faf6f2

9. RGPD

- Nous utilisons uniquement des cookies essentiels (authentification) et ne collectons aucune donnée supplémentaire.
- Nous utilisons le service respectueux de la vie privée plausible.io pour le suivi des visites. Plus d'infos sur Plausible et le RGPD :<https://plausible.io/data-policy>
- Politique de confidentialité : <https://ecomobilis.be/page/privacy-policy>
- Conditions générales : <https://ecomobilis.be/page/terms-of-use>

Un utilisateur peut supprimer immédiatement son compte sans intervention manuelle. Ses données personnelles sont effacées instantanément.

10. Panneau d'administration

Il existe un dépôt séparé uniquement pour l'administration : <https://github.com/Telraam-Rear-Window-BV/ecomobilis-admin>

Pourquoi Filament ?

Laravel Filament est un panneau d'administration moderne et une boîte à outils pour Laravel qui facilite la création de tableaux de bord internes et d'applications back-office rapides et élégants. Il offre une interface claire et intuitive avec des fonctionnalités CRUD prêtes à l'emploi, des générateurs de formulaires et des tables, tout en restant hautement personnalisable. En l'exécutant dans un dépôt séparé, nous gardons l'application principale propre et modulaire tout en profitant des puissantes fonctionnalités de Filament pour gérer efficacement les données, les utilisateurs et la configuration système.

11. Applications natives iOS et Android

En plus de l'application web, il existe aussi une application légère pour iOS et Android : <https://github.com/Telraam-Rear-Window-BV/ecomobilis-mobile-app>

Elle n'a pas d'authentification intégrée mais constitue une bonne introduction au projet. Nous avons utilisé Expo, un framework basé sur React Native pour développer des applications mobiles natives. Expo propose un flux de travail unifié pour iOS et Android. En la maintenant dans un dépôt séparé, nous découplons l'application mobile du backend Laravel, garantissant une séparation claire des responsabilités. Cela permet à l'application Expo de se concentrer uniquement sur l'expérience mobile fluide et performante, tout en consommant les API de nos services principaux. Résultat : un développement plus rapide, une maintenance plus simple et la flexibilité d'évoluer indépendamment du backend.

12. Données

Liste des jeux de données utilisés :

- Nous utilisons les cartes et localisations d'OpenStreetMap via Locationiq

Liste des jeux de données générés/ouverts :

- Les jeux de données intéressants sont ouverts via l'API

Portail ODWB :

Les données sont ouvertes et disponibles via l'API. Les données en direct Telraam provenant du réseau seront transmises à l'ODWB. Ces données contiennent une entité GeoJSON avec la géométrie des segments sous forme de MultiLineString et le nombre de catégories principales à côté du v85 pour une heure donnée (UTC). Cette méthode vous permet d'obtenir toutes les informations nécessaires pour créer une carte Telraam. Les détails et les données historiques peuvent être demandés par appels successifs par rue ou par identifiant d'installation/d'instance. En attente d'approbation par l'ODWB pour le moment.